# Projekt 2 – Be Aware

Security-Awareness-Programm & Gophish Proof of Concept

**Kunde:** Revolut (Fintech)  **Zielgruppe:** Entwickler
**Autor:** Roman Rubanyk  Modul 2 · Woche 3  **Datum:** 01.08.2026

---

Revolut ist eine schnell wachsende Digitalbank mit einer technikaffinen Belegschaft. Das *klassische, einmal jährliche Pflichttraining* greift hier nicht. Der Head of Security nennt einen konkreten Beinahe-Vorfall – ein Social-Engineering-Versuch über Slack, der fast einen Junior-Entwickler zur Weitergabe eines Test-Credentials verleitet hätte – sowie das Kernrisiko der Entwickler: Sie leben auf GitHub, testen ständig neue Tools, und ein einziger geleakter API-Key kann verheerend sein. Dieser Report umfasst die beiden Deliverables: einen visuellen Awareness-Workshop (Deliverable 1) und einen lauffähigen Gophish Proof of Concept (Deliverable 2).

## Deliverable 1 – Awareness-Workshop

Da dies ein **Workshop** ist, steht das visuelle Erkennen im Mittelpunkt: Die Teilnehmer sehen genau den Angriff, der sie im Alltag treffen würde, und markieren gemeinsam die Warnsignale – in E-Mail und auf der Login-Seite. Kein Vortrag, sondern „Spot the signs".

### Zielgruppe & Ziel

**Zielgruppe:** die Entwickler von Revolut – bevorzugtes Angriffsziel, da sie öffentlich auf GitHub arbeiten und laufend neue Tools und Zugangsdaten (API-Keys, Test-Credentials) verwenden. **Risikoszenario:** Credential-Phishing über eine gefälschte Entwickler-Tool-Einladung („Codex Workspace Developer Preview"), die zur Anmeldung mit den Arbeits-Zugangsdaten auffordert. **Ziel des Workshops:** sich niemals über einen Link aus einer E-Mail anmelden, sondern den Dienst selbst aufrufen und verdächtige Nachrichten melden.

### 1) Die Phishing-Mail erkennen

Wir zeigen die Original-E-Mail und lassen die Gruppe die Warnsignale finden. Jede rote Markierung ist ein Signal – einzeln verdächtig, zusammen ein klarer Alarm:

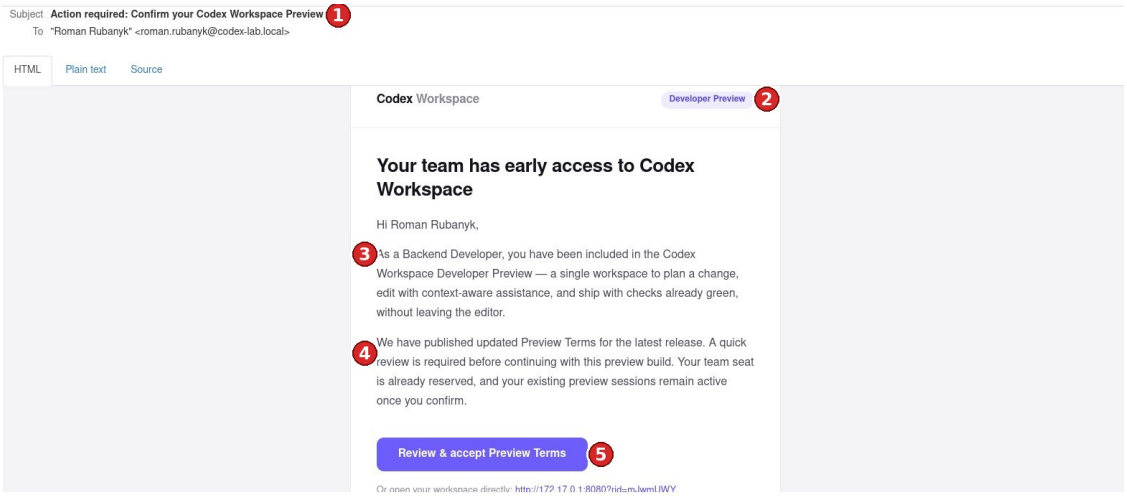



*Abb. 1 – Phishing-E-Mail mit markierten Warnsignalen*

1 **Betreff „Action required" –** künstlicher Handlungsdruck, damit man schnell und unüberlegt reagiert.

2 **Unerwartete „Developer Preview"-Einladung –** Zugang, den niemand angefordert hat; die Absenderdomain (codex-preview.com) passt nicht zur eigenen Firmendomain.

3 **Personalisierte Anrede „As a Backend Developer" –** gezielt auf die Rolle zugeschnitten, damit die Mail legitim wirkt.

4 **Vorwand „updated Preview Terms … required" –** ein plausibel klingender Policy-Grund, der zum sofortigen Handeln drängt.

5 **Button = Login über einen E-Mail-Link –** zusätzlich ein roher IP-Link (http://172.17.0.1:8080). Seriöse Dienste fordern das nicht.

## 2) Die gefälschte Login-Seite erkennen

Der Button führt auf eine überzeugende Login-Seite. Auch hier markieren wir gemeinsam, woran man den Betrug erkennt:

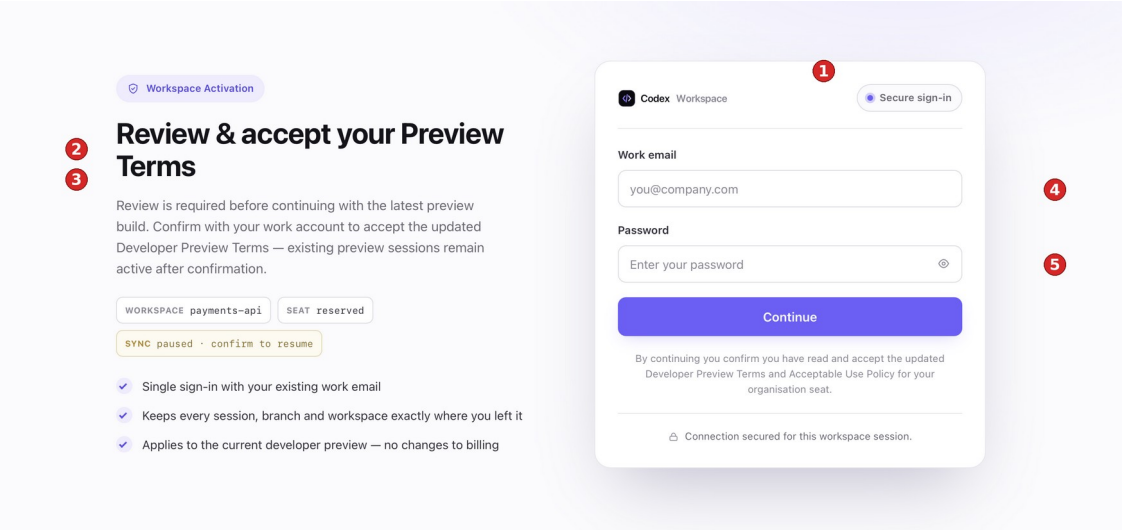



*Abb. 2 – Gefälschte „Review & accept your Preview Terms"-Seite mit markierten Warnsignalen*

1 **Look-alike-Branding & „Secure sign-in" –** täuscht Vertrauen vor, obwohl die Seite über einen E-Mail-Link geöffnet wurde.

2 **Autorität & Druck „Review is required" –** die Anmeldung wird als verpflichtend dargestellt.

3 **Vorwand „accept the updated Preview Terms" –** rechtfertigt scheinbar die Passwortabfrage.

4 **Abfrage der Arbeits-E-Mail –** das Ziel sind Unternehmens-Zugangsdaten, nicht ein privates Konto.

5 **Passwort-Eingabe auf einer verlinkten Seite –** genau das will der Angreifer; die Zieldomain in der Adressleiste stimmt nicht mit dem echten Dienst überein.

## 3) Echt vs. Fake – die schnelle Faustregel

| Merkmal | Legitim | Phishing (dieses Beispiel) |
|---|---|---|
| **Anmeldung** | Dienst selbst im Browser öffnen | Login über Button/Link in der E-Mail |
| **Absender / Domain** | bekannte, verifizierte Domain | fremde Domain / rohe IP (codex-preview.com, 172.17.0.1) |
| **Ton** | sachlich, kein Zeitdruck | Dringlichkeit & Exklusivität („required", „reserved") |
| **Passwort** | nie über einen Mail-Link eingeben | Passwortabfrage auf verlinkter Seite |
| **Passwort-Manager** | füllt auf echter Domain aus | füllt auf Look-alike-Domain NICHT aus – ein Warnsignal |

## Workshop-Ablauf (ca. 10 Minuten)

- **Hook (1 Min):** „Wer hat diese Woche eine Tool-Einladung oder ein Beta-Angebot bekommen?"
- **Live-Demo (2 Min):** die echte Mail und Login-Seite aus dem Gophish-PoC zeigen.
- **Spot-the-Signs (4 Min):** die Gruppe markiert gemeinsam die Warnsignale (Abb. 1 & 2) – wie in einem Quiz.
- **Echt vs. Fake (2 Min):** die Faustregel-Tabelle durchgehen.
- **Habits (1 Min):** Absender prüfen, URL prüfen, Passwort-Manager, phishing-resistente MFA/Passkeys, melden.

### Key Takeaway

> **„Kein Login über einen Link aus einer E-Mail.** Im Zweifel innehalten, den Absender prüfen und den Dienst selbst öffnen.“ Auch die Besten können getäuscht werden – das Innehalten ist die wirksamste Verteidigung.

## Deliverable 2 – Gophish Proof of Concept

Dieser Teil zeigt die technische Umsetzung: wie die Kampagne aufgebaut, versendet und ausgewertet wurde.

### Technischer Aufbau

Der PoC läuft vollständig lokal und isoliert. **Gophish** (Open-Source-Phishing-Framework) wurde als Docker-Container aufgesetzt und über das Web-Dashboard verwaltet. Als Mailserver dient **MailHog**, ein Test-SMTP-Server, der alle ausgehenden Mails abfängt, ohne real etwas nach außen zu senden.

- **Kosten:** Open Source, keine Lizenzkosten – die geforderte günstige Alternative.
- **Datenschutz:** Passwort-Erfassung ist deaktiviert; protokolliert wird nur, dass ein Formular abgeschickt wurde – nicht der Wert.

### Kampagne: Landing Page & Awareness-Redirect

Für die Zielgruppe „Entwickler“ wurden ein E-Mail-Template (Abb. 1) und eine Landing Page (Abb. 2) erstellt. Die Landing Page beginnt mit einer überzeugenden Produktseite, die Ton und Qualität einer modernen AI-Entwicklerplattform imitiert, bevor die Login-Seite die Zugangsdaten abfragt.

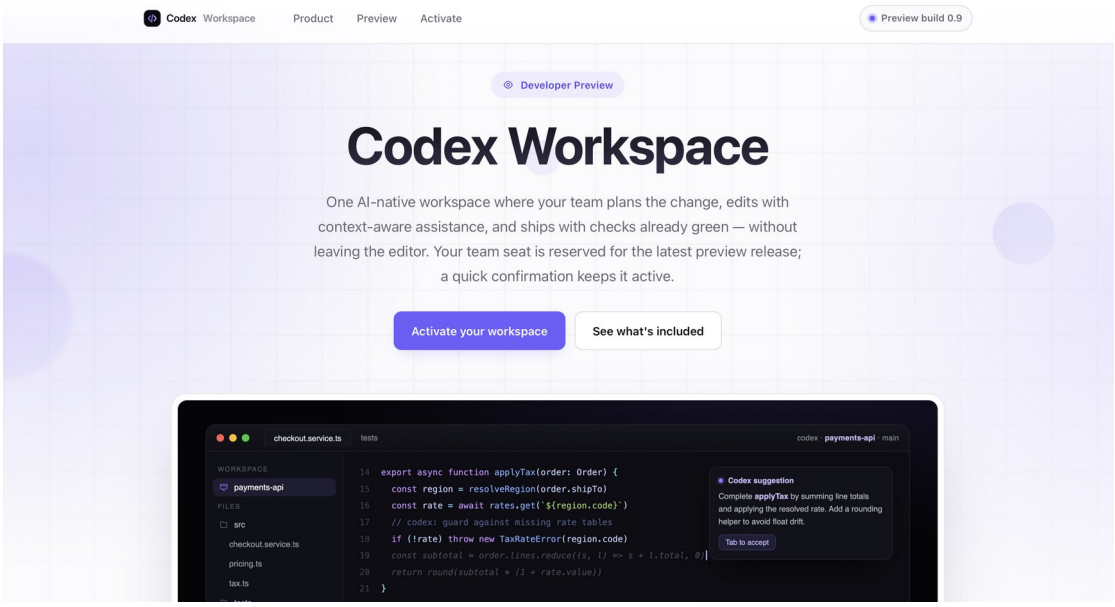



*Abb. 3 – Gefälschte Produktseite „Codex Workspace“ (Look-alike), Einstieg der Landing Page*

Nach dem Absenden wird die Person auf eine **Awareness-Debrief-Seite** umgeleitet, die transparent erklärt, dass es eine autorisierte Simulation war, was passiert ist und wie man sich schützt – so wird der „Fehler“ sofort zum Lernmoment, ohne Bloßstellung.

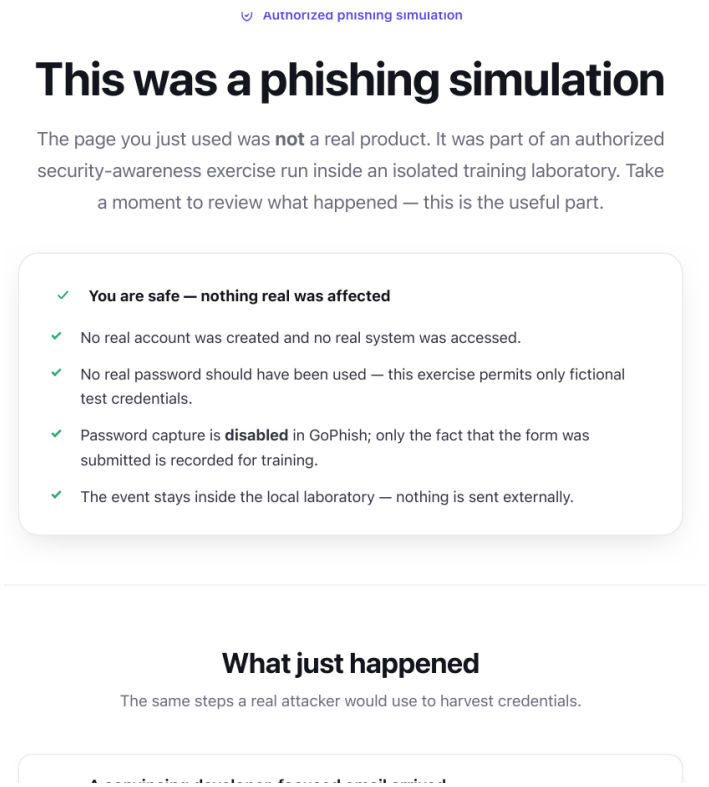



*Abb. 4 – Awareness-Debrief nach dem Absenden*

## Zielpersonen & Zustellung

Als Zielgruppe wurden vier fiktive Entwickler-Profile angelegt (Domain codex-lab.local – keine realen Adressen): Alex Weber (Senior Developer), Dennis Fischer (DevOps Engineer), Markus Klein (Software Engineer) und Roman Rubanyk (Backend Developer). Alle vier Mails wurden über MailHog erfolgreich zugestellt.

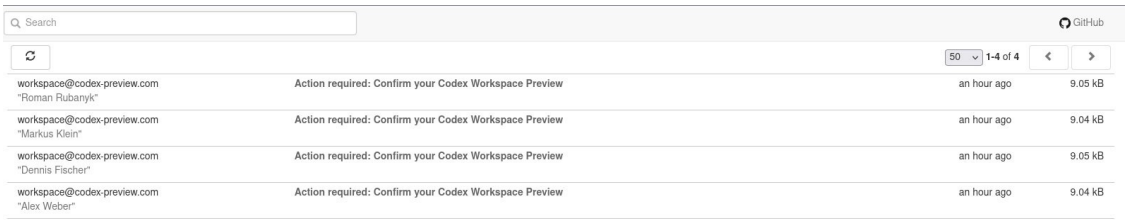



*Abb. 5 – MailHog-Inbox: alle 4 Simulations-Mails abgefangen*

## Ergebnisse (Live-Dashboard)

Das Gophish-Dashboard verfolgt jede Interaktion in Echtzeit und liefert die Kennzahlen für ein datengetriebenes Programm.

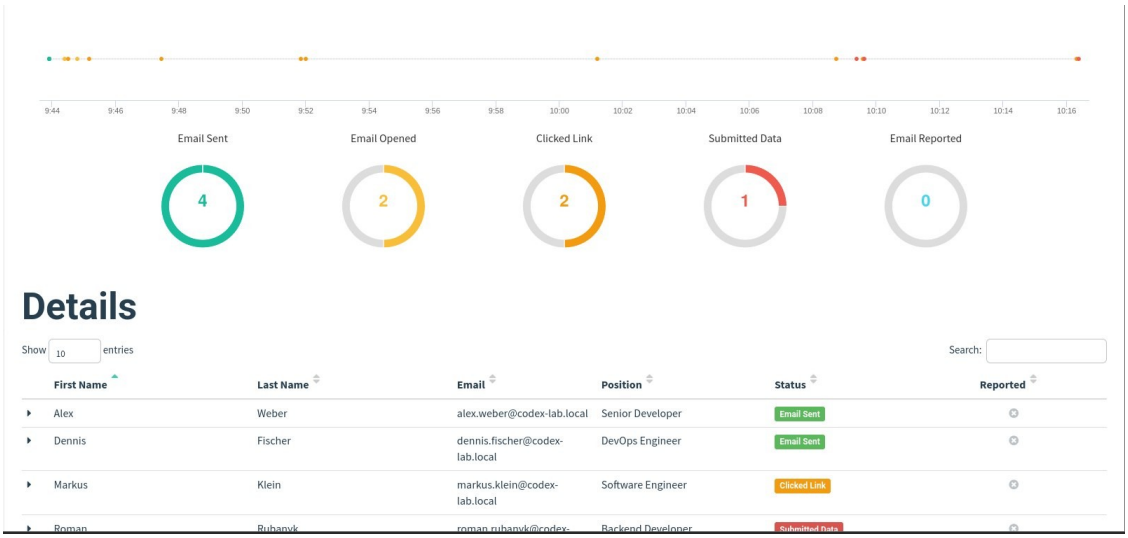


*Abb. 6 – Gophish-Dashboard mit Zeitleiste und Trichter (4 / 2 / 2 / 1 / 0)*

| Metrik | Wert | Bedeutung |
|---|---|---|
| **E-Mails versendet** | 4 | Alle Zielpersonen über MailHog zugestellt |
| **E-Mails geöffnet** | 2 | Nachricht wurde wahrgenommen |
| **Link geklickt** | 2 | Landing Page aufgerufen |
| **Daten abgesendet** | 1 | Login-Formular abgeschickt (kritisch) |
| **Als Phishing gemeldet** | 0 | Niemand hat die Mail gemeldet |

**Interpretation:** Die Hälfte öffnete die Mail, zwei klickten, eine Person schickte das Login-Formular ab – in einem realen Angriff wäre dieses Credential kompromittiert. Bemerkenswert: 0 Reported. Diese fehlende Melde-Reflexe ist die eigentliche Lücke und begründet, den „Melden"-Prozess im Workshop zu betonen.

### Sicherheit, Ethik & Mehrwert

- Reine Simulation: keine echten Mails, keine realen Revolut-Adressen, keine echten Zugangsdaten – alles bleibt in der isolierten Laborumgebung.
- Passwort-Werte werden nicht gespeichert; erfasst wird nur die Aktion selbst.
- **Wert für das Programm:** wiederholbare, gezielte Simulationen und messbare Daten (Öffnungs-, Klick-, Absende- und Melderaten) pro Zielgruppe – kostenlos, in-house und compliance-freundlich.

## Fazit

Workshop und Gophish-PoC greifen ineinander: Der PoC liefert das reale, „native" Angriffsbeispiel und die Kennzahlen, der Workshop übersetzt beides in eine konkrete Verhaltensänderung für Revolut-Entwickler. Die zentrale Botschaft bleibt einfach – **kein Login über E-Mail-Links, im Zweifel innehalten und melden**. Statt eines teuren Jahres-Pflichttrainings entsteht ein wiederholbares, messbares und zielgruppengerechtes Programm auf Open-Source-Basis.